

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals and Data Structures	Course Code: CMPE-112L
Assignment Type: Lab Report	Dated: 7th March, 2026
Semester: 1st	Session: 2026
Lab/Project/Assignment #: 13	CLOs to be covered: CLO2
Lab Title: MAP, Filter and Reduce	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO2	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 13: MAP, Filter and Reduce

Lab Goals/Objectives:

To understand and use the map() function together with lambda expressions to apply a transformation to every element of an iterable.

Equipment Required:

Python 3, VS Code / PyCharm

Theory:

The `map()` function applies a given function to every item of an iterable (such as a list) and returns a map object, which can be converted to a list. It is often used together with lambda functions for concise, one-line transformations.

Lab Work:**Task 1: Map with Lambda**

```
a= [1,2,3,4]
b = list(map(lambda x: x**2,a))
print(b)
```

Result:

```
[1, 4, 9, 16]
```

Task 2: Square all numbers in a list

```
numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x ** 2, numbers))
print(squared)
```

Result:

```
[1, 4, 9, 16, 25]
```

Task 3: Convert list of temperatures from Celsius to Fahrenheit

```
celsius = [0, 20, 37, 100]
fahrenheit = list(map(lambda c: (c * 9/5) + 32, celsius))
print(fahrenheit)
```

Result:

```
[32.0, 68.0, 98.6, 212.0]
```

Task 4: Get length of each string in a list

```
words = ["Python", "Java", "C++", "JavaScript"]
lengths = list(map(lambda w: len(w), words))
print(lengths)
```

Result:

```
[6, 4, 3, 10]
```

Task 5: Filter even numbers from a list

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)
```

Result:

```
[2, 4, 6, 8, 10]
```

Task 6: Filter words longer than 5 characters

```
words = ["cat", "elephant", "dog", "giraffe", "bird", "butterfly"]
long_words = list(filter(lambda w: len(w) > 5, words))
print(long_words)
```

Result:

```
['elephant', 'giraffe', 'butterfly']
```

Task 7: Filter numbers greater than 10

```
numbers = [5, 12, 8, 15, 3, 20, 7]
greater_than_10 = list(filter(lambda x: x > 10, numbers))
print(greater_than_10)
```

Result:

```
[12, 15, 20]
```

Task 8: Filter strings that start with 'A' or 'a'

```
names = ["Alice", "Bob", "Charlie", "Amanda", "David", "Alex"]
a_names = list(filter(lambda n: n[0].lower() == 'a', names))
print(a_names)
```

Result:

```
['Alice', 'Amanda', 'Alex']
```

Task 9: Find product of all numbers

```
from functools import reduce
numbers = [1, 2, 3, 4, 5]
product = reduce(lambda x, y: x * y, numbers)
print(product)
```

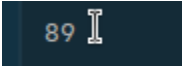
Result:

```
120
```

Task 10: Find maximum number in a list

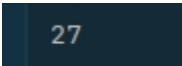
```
numbers = [23, 45, 12, 67, 34, 89, 5]  
max_num = reduce(lambda x, y: x if x > y else y, numbers)  
print(max_num)
```

Result:

A dark-themed terminal window showing the output of the code. The number 89 is displayed in a light blue font, followed by a cursor icon.**Task 11: Count total characters in all strings**

```
words = ["hello", "world", "python", "programming"]  
total_chars = reduce(lambda x, y: x + len(y), words, 0)  
print(total_chars)
```

Result:

A dark-themed terminal window showing the output of the code. The number 27 is displayed in a light blue font.**Task 12: Find the sum of squares of all numbers**

```
numbers = [1, 2, 3, 4, 5]  
sum_of_squares = reduce(lambda x, y: x + (y ** 2), numbers, 0)  
print(sum_of_squares)
```

Result:

A dark-themed terminal window showing the output of the code. The number 55 is displayed in a light blue font.**Conclusions:**

The map() function was combined with a lambda expression to transform every element of a list, demonstrating functional-style iteration.